Photon Fusion 101-Unity

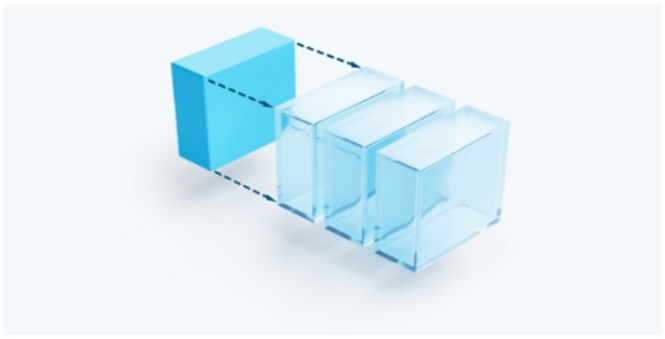
Network Object Synchronization

Instructor: Demir Bayık
Used Unity Version: 2022.1.23f1
Used Photon Fusion 1
Fusion Compatible Unity Versions: 2020.3 LTS and newer.
Github Repository

Why Use Photon Fusion?

Tick-Based Simulation

Core feature of stable and accurate networking. Foundation for client side prediction and snapshot interpolation.

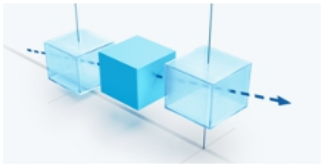


Client-Side Prediction

Give players an instant response to their own inputs without giving up server authority, even under high latency and network loss.

High Performance

Handle thousands of networked object over hundreds of client connections, including complex interest management.



Snapshot Interpolation

Provide smooth visual rendering of remote objects for a client, even during bad networking conditions.

Lag Compensation

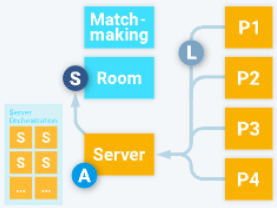
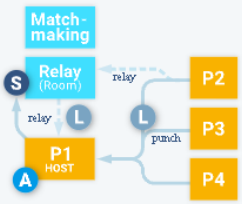
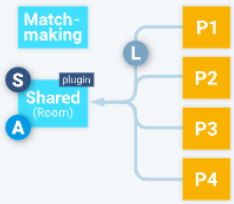
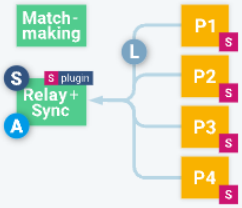
Compensate for the delay between the client firing his weapon and the server registering the potential hit.

Replication Systems

Per object full consistency or eventual consistency.

Photon Fusion saves you lots of time without spending time for server side and client side implementations. You can script your own Component and synchronize it across clients easily. Other Network Api's like Mirror offering either dedicated server or client host implementations, which means it will cost or client host advantage gonna happen while Photon Fusion offers no cost with no client-host advantage. You can also make large-scale projects with Photon Fusion.

Network Topologies:

DEDICATED **SERVER**	CLIENT **HOST**	**SHARED** AUTHORITY	**DETERMINISTIC**
Dedicated server with public IP (headless Unity instance)	One Player is the „host". All other Players connect to him.	Room has state authority and optional plugin (custom code).	Quantum: Each client runs a deterministic simulation.
			
Authority			
++ Server has full authority	-- Host has full authority Hacking/cheating possible	/ Room has state authority but clients control state of objects assigned to them	++ Server has input authority and can referee sim
State			
++ Server migration possible	++ Turnkey host migration	++ Room is keeping the full game state	++ Room is keeping the full game state
Latency			
++ Low latency: Direct connection from client to server	/ Latency good (punch) or relatively bad (relay) Quality depends on host	++ Direct connection to room in region	++ Direct connection to room in region (edge)
QoS			
++ Stability of the game engine	- Depends on players/hosts	+ High stability + uptime	+ High stability + uptime
Cost			
-- Dedicated server orchestration	++ Hosted by players + Photon Cloud	++ Cost efficient	++ Cost efficient

Dedicated Server: This is the best way for preventing cheats, no advantage among clients and no host clients are used in this topology. Games like CS2, R6 and DayZ are using this topology.

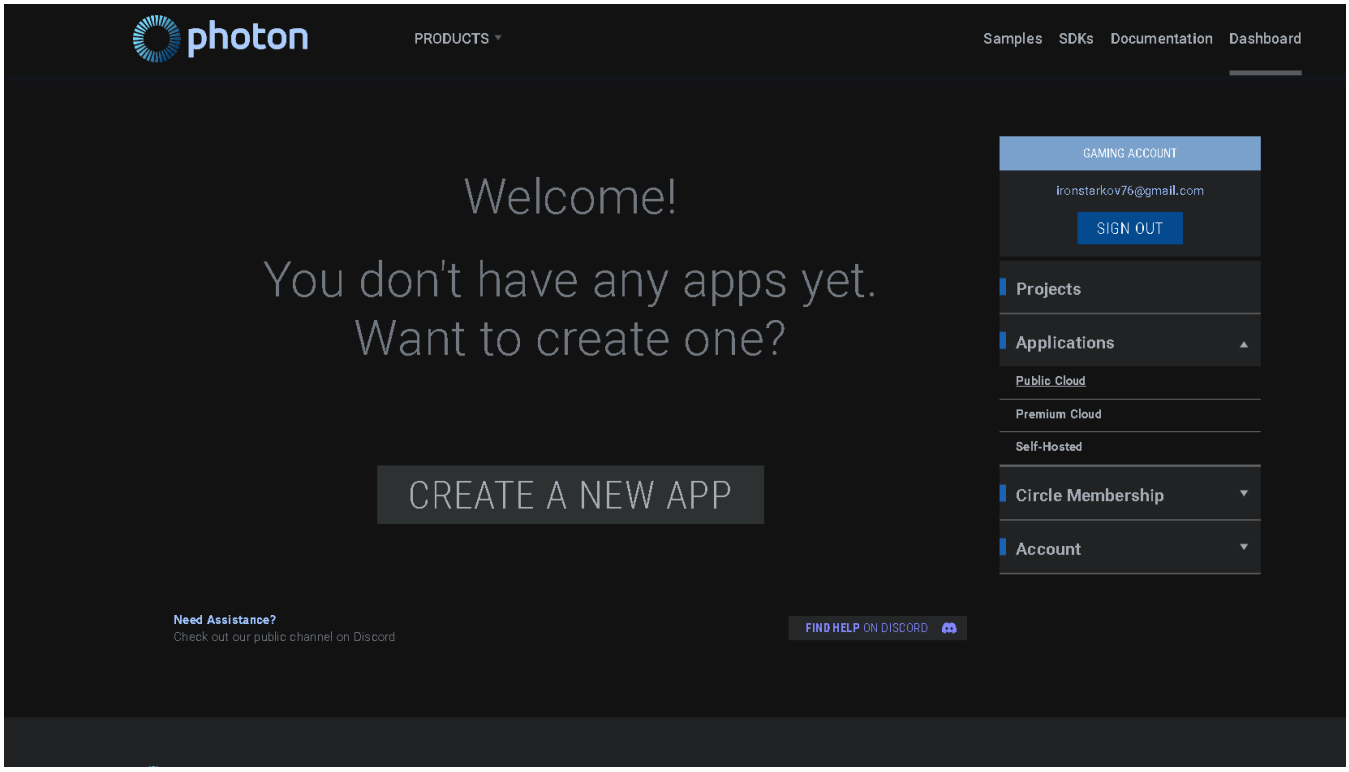
Client Host: Cost free but high chance of hacking probability also giving reaction time advantage to the host client. For an instance Gta 5 Online has this topology and that is one of the reasons why there are lots of modders in PC version. This topology also causes Ip Address leakage.

Fusion's Shared Mode: Free up to 20 CCU or 100 CCU. No advantage among clients. Every client has full ownership of their Network Objects. Today Shared Mode is what we use.

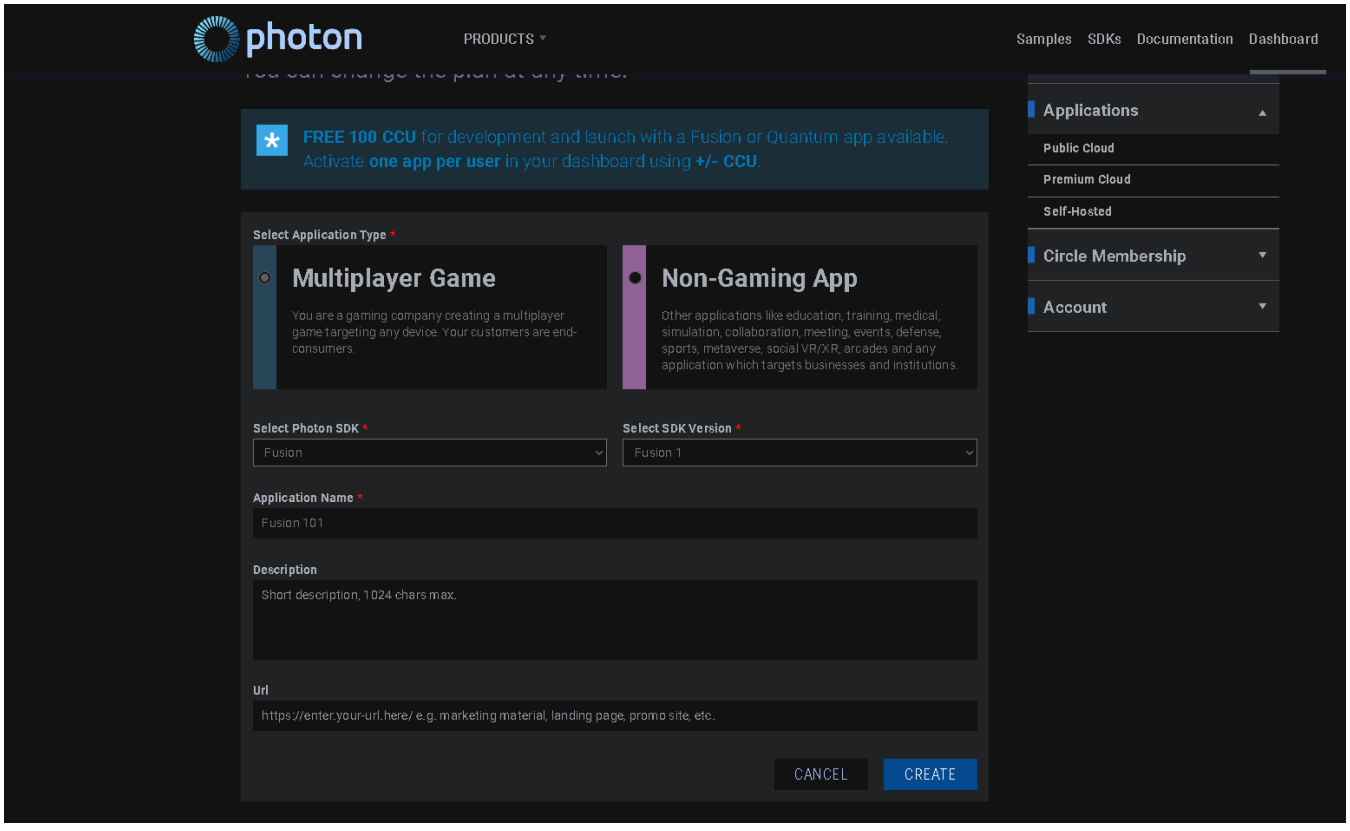
Registering/Logging in to Photon

https://www.photonengine.com/fusion

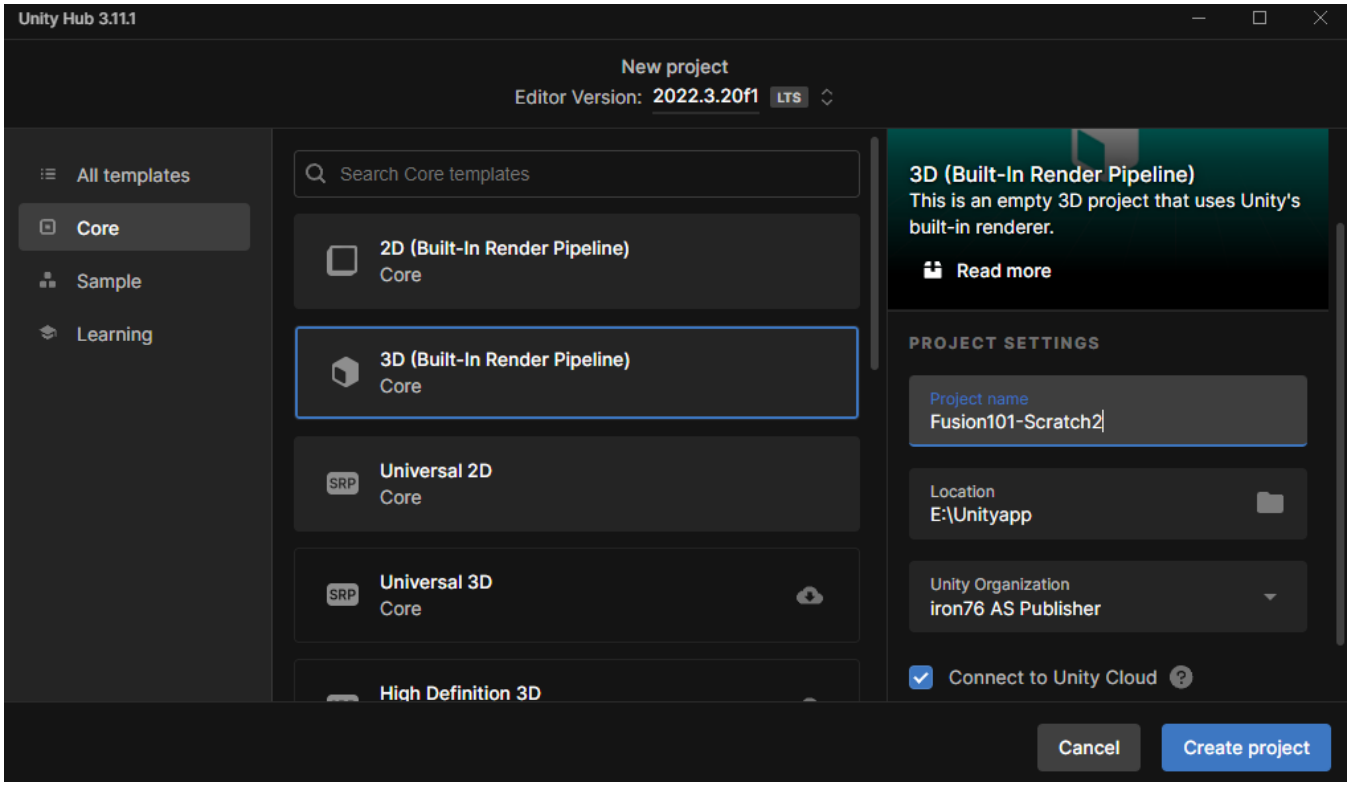
#1: Head to the dashboard



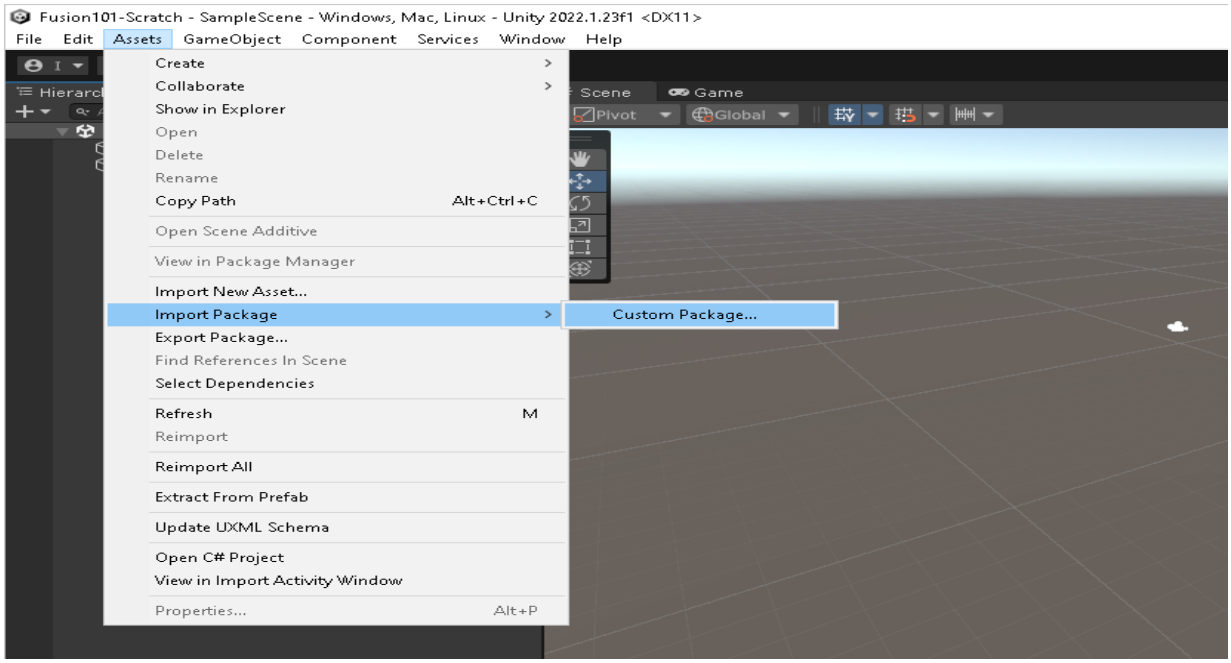
#2: Create New App



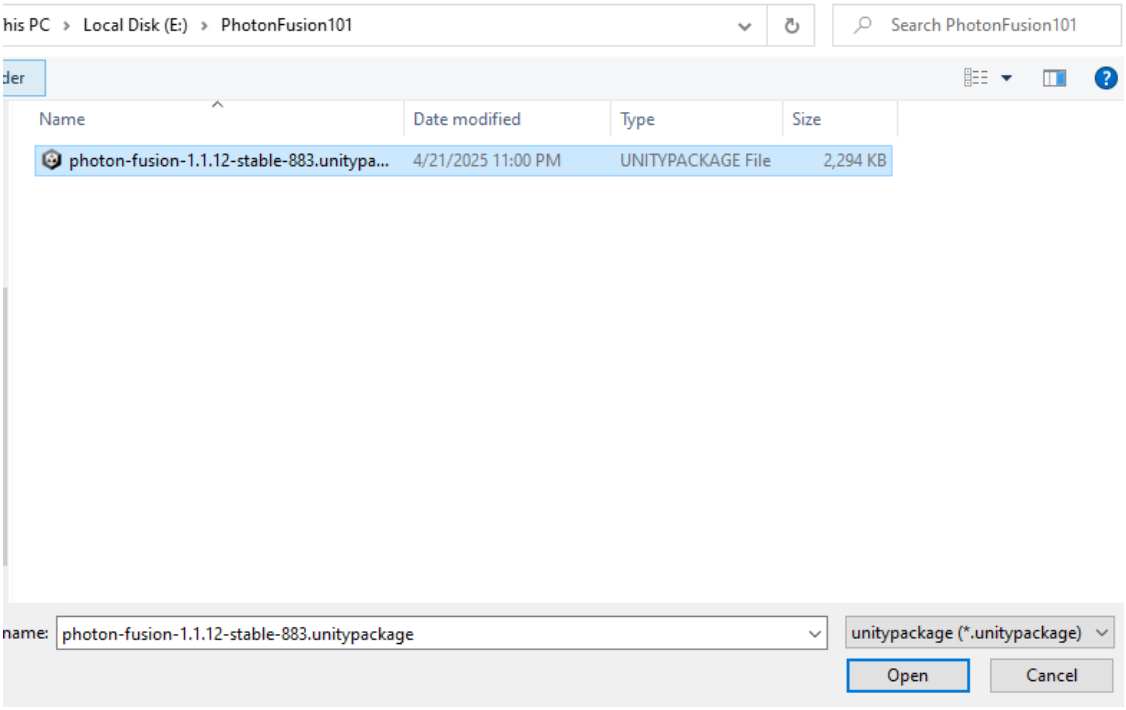
#3,1: Starting With Blank Unity Project And



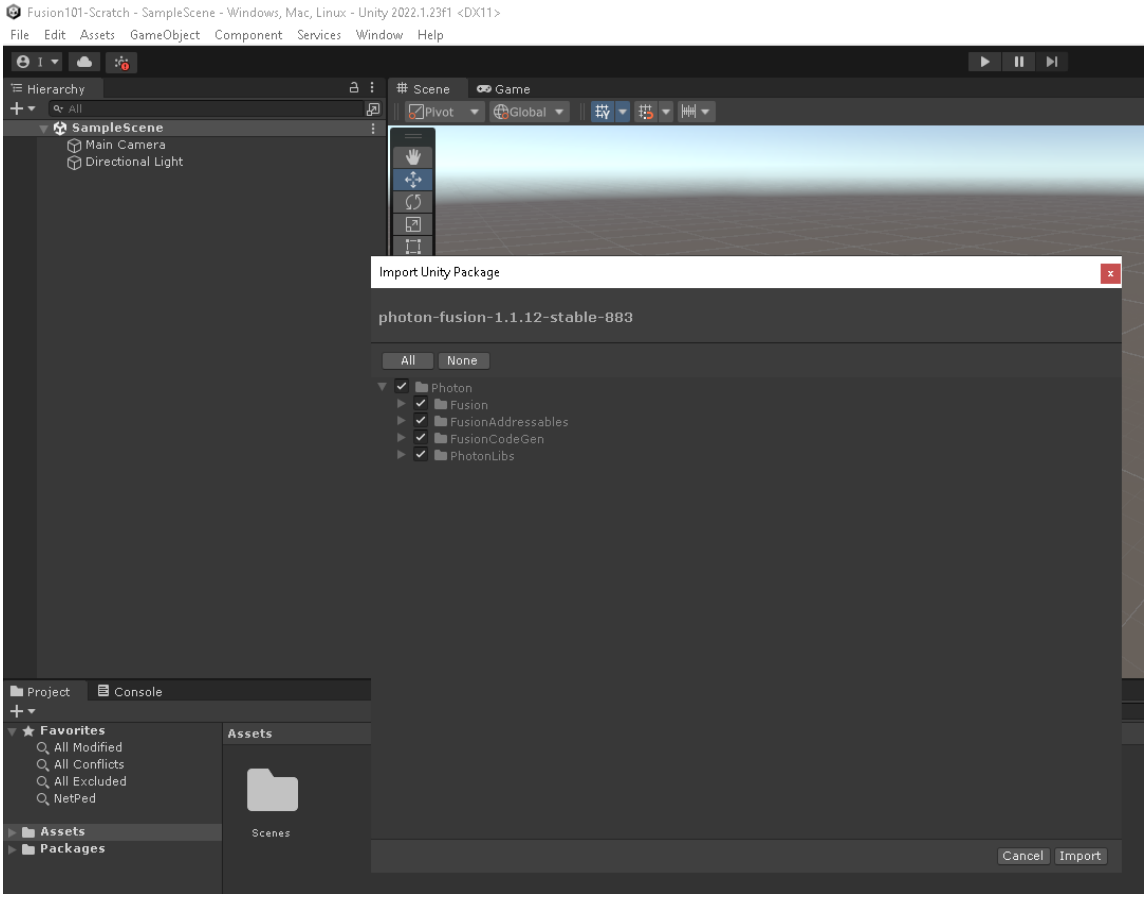
#3,2: Importing Photon Fusion



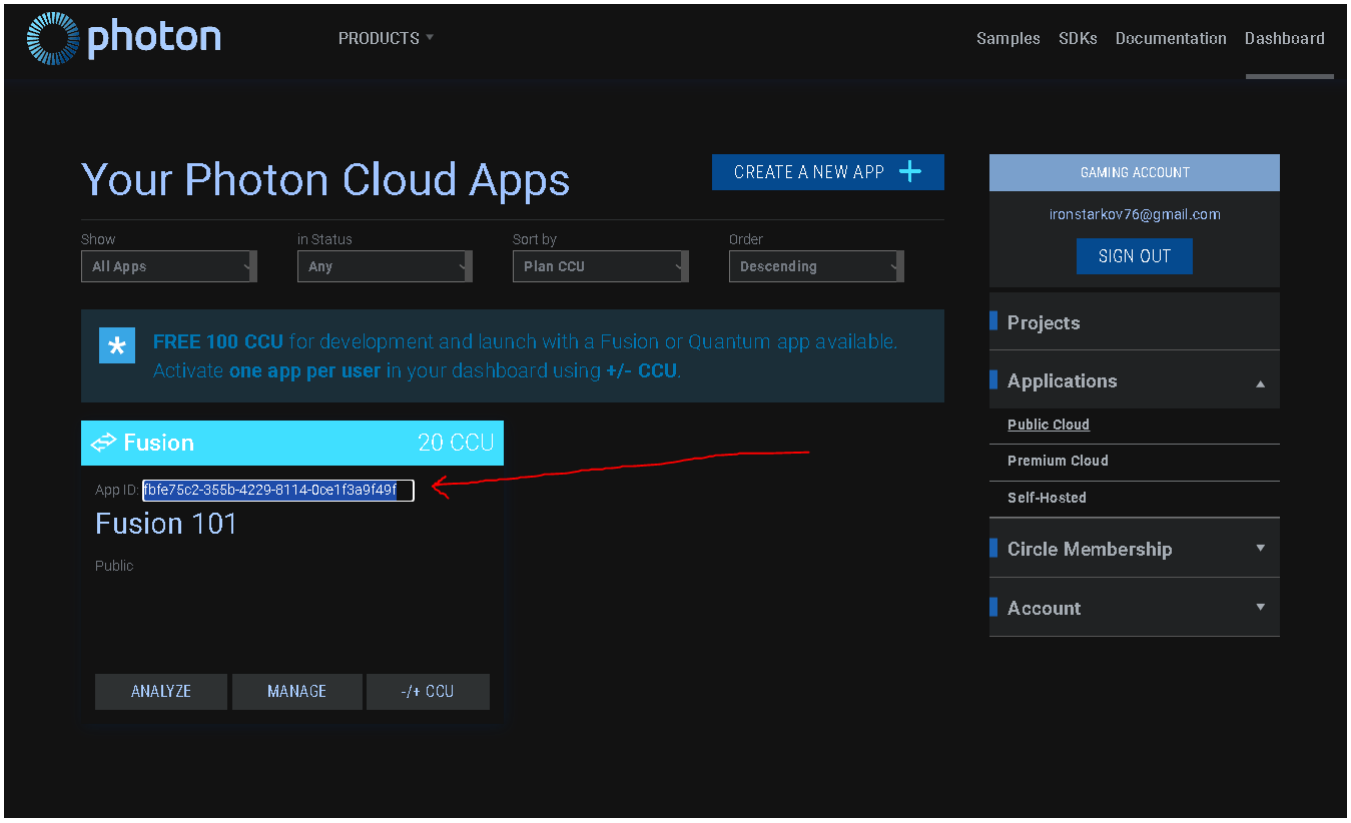
#3,3: Select Fusion From Your Directory



#3,4: Import All Packages

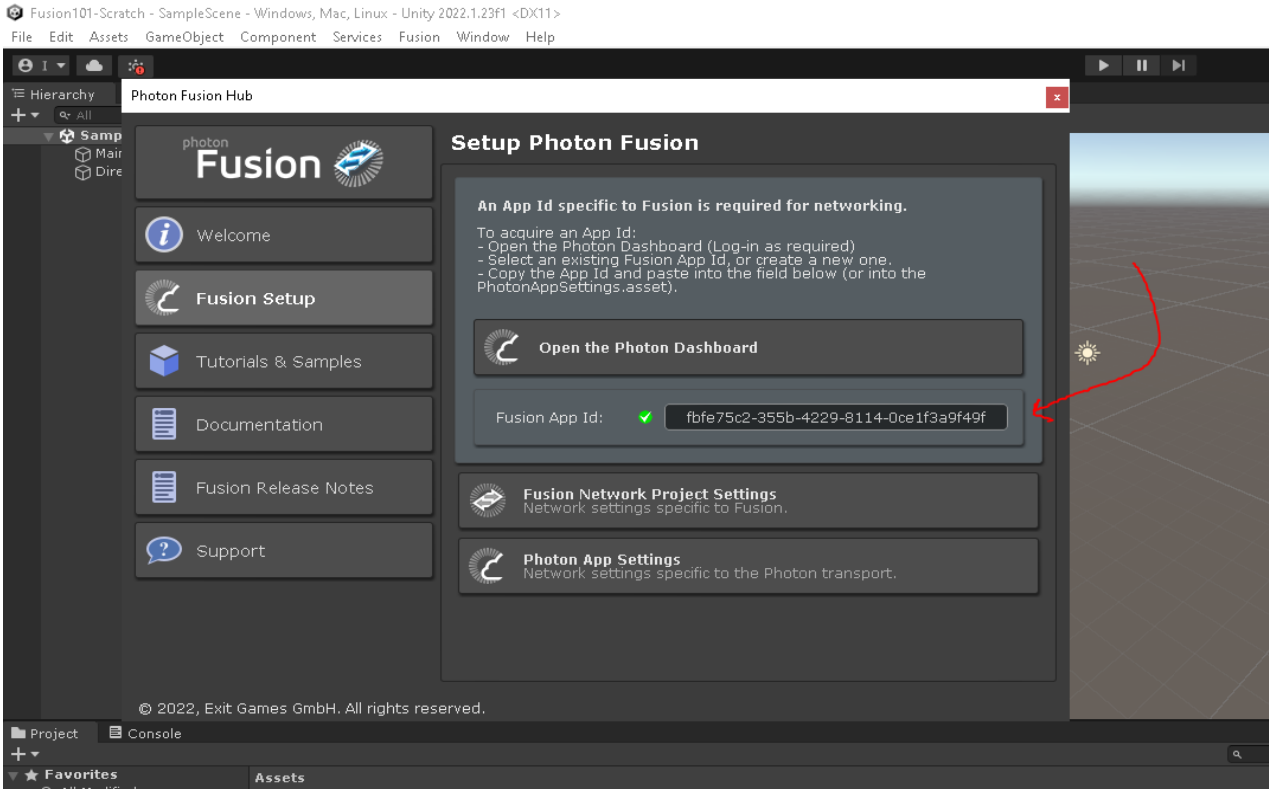


#4: Copy Your Project AppID From The Dashboard

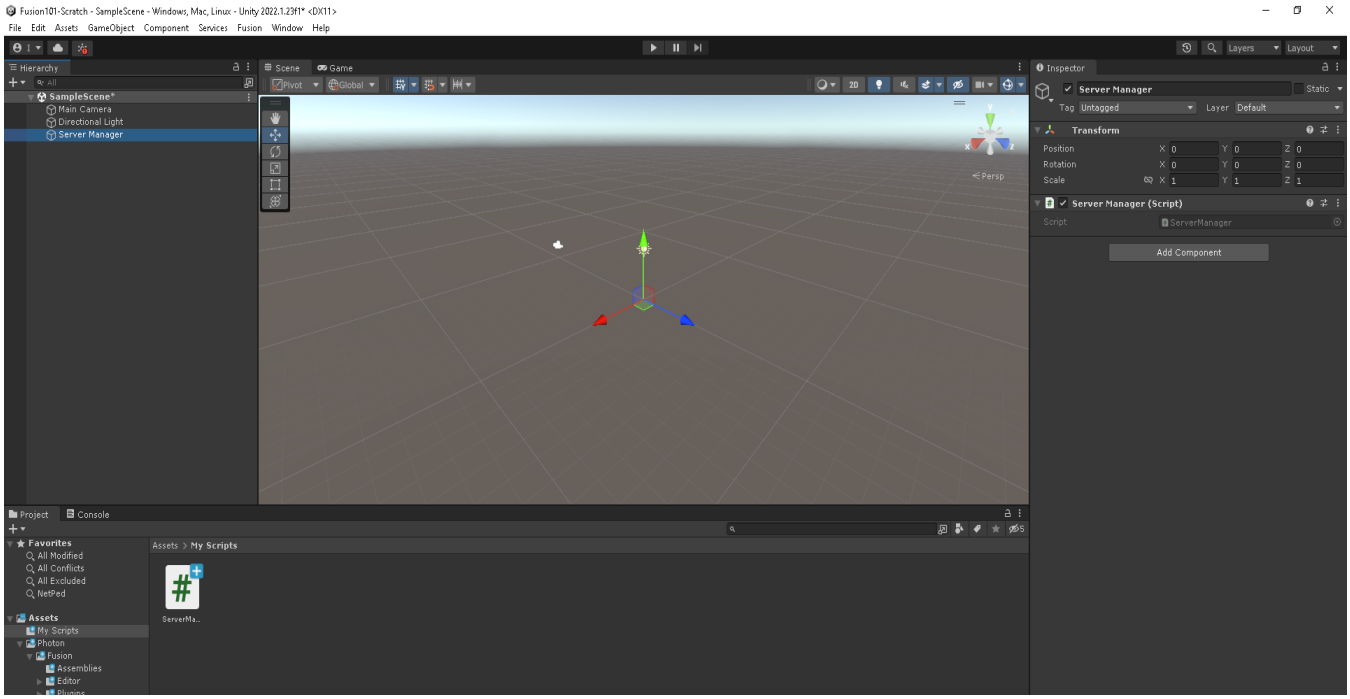


#5: Paste Your Project AppID To Fusion App Id

Also you can Alt+F5 to open Photon Fusion Hub.

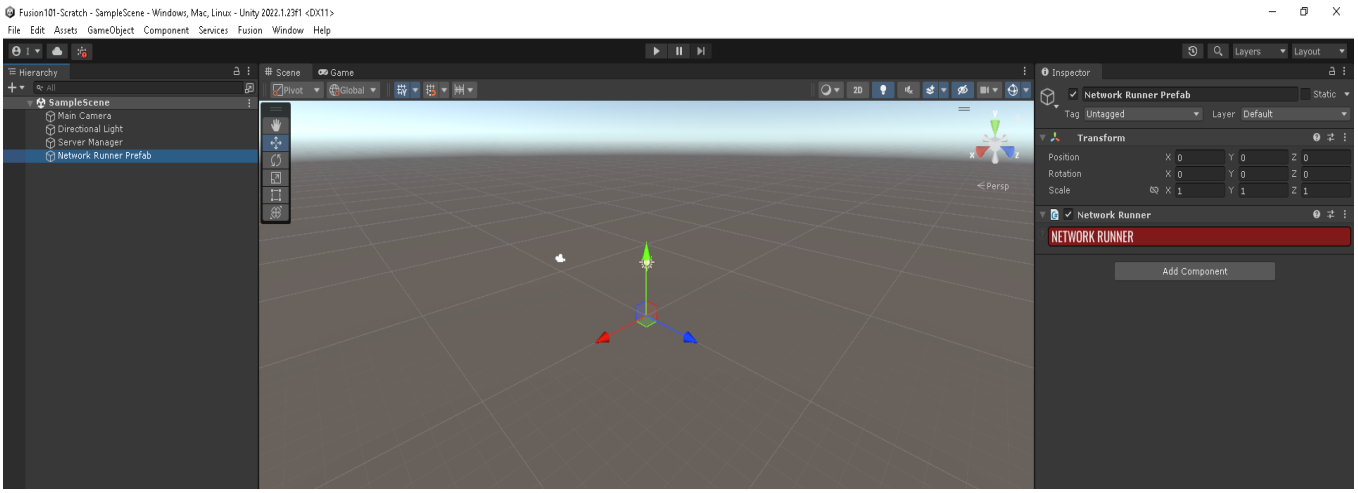


#6: Create Server Manager And ServerManager.cs



Create a new GameObject called Server Manager and Create a new Script called ServerManager add the script to our new Server Manager GameObject.

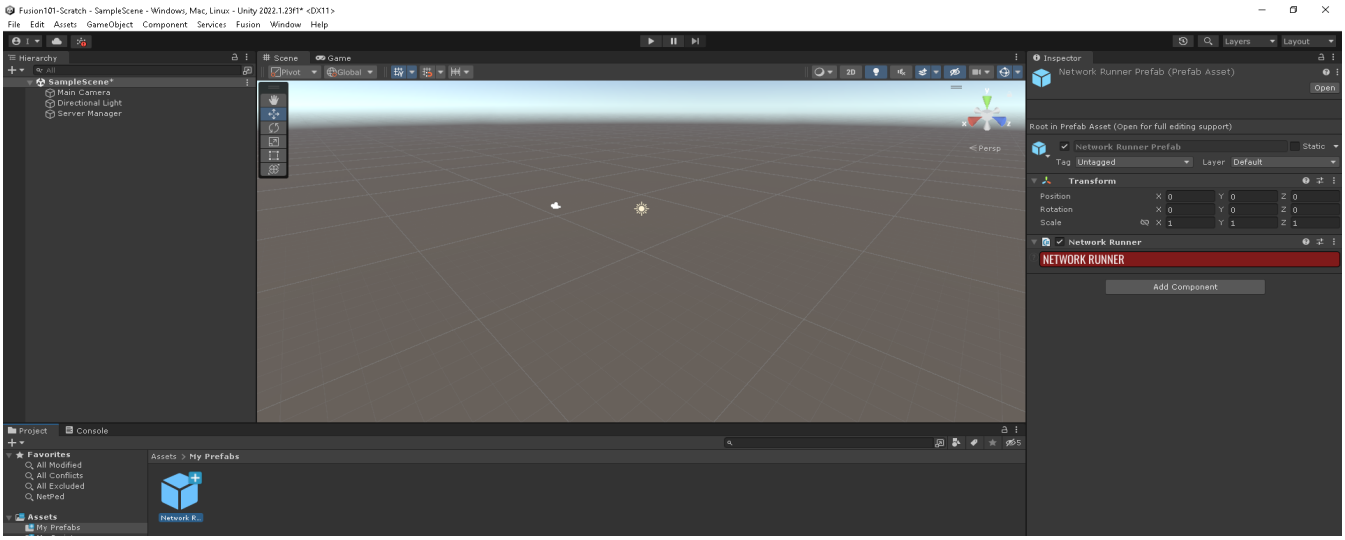
#7,1: Create Network Runner



Create new GameObject called Network Runner Prefab And Add Component called Network Runner.

#7,2: Make Network Runner Prefab

Drag and drop the Network Runner from the scene to a folder to make it prefab. And Delete it from the scene



Network Runner is Fusion's Server Handler must Instantiate Network Runner GameObject. Function StartGame for connecting to server is going to be called from this Network Runner Component. It is an MonoBehaviour component, so to keep it alive we have to use a GameObject with it.

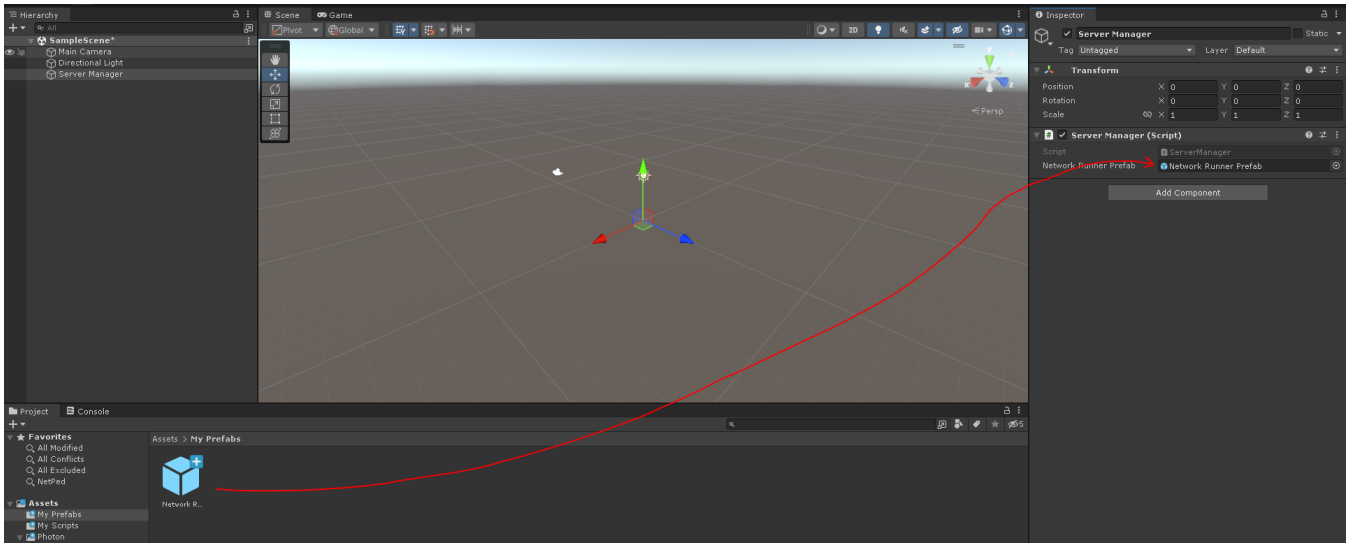
#8,1: First Time Scripting ServerManager

ServerManager.cs

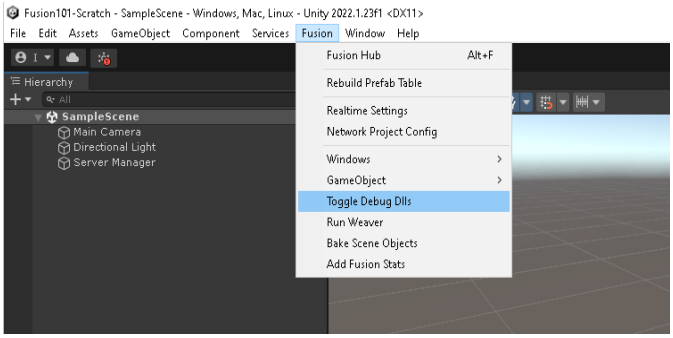
```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using System;
using Fusion;//IMPORTANT!
using Fusion.Sockets;//IMPORTANT!

public class ServerManager : MonoBehaviour, INetworkRunnerCallbacks
{
    [SerializeField] GameObject NetworkRunnerPrefab;
    public static NetworkRunner NetworkRunner;

    void Start()
    {
        NetworkRunner = Instantiate(NetworkRunnerPrefab).GetComponent<NetworkRunner>();//Creating Network Runner
        NetworkRunner.AddCallbacks(this);
        NetworkRunner.StartGame(args: new StartGameArgs() //Connecting to the server and the session
        {
            GameMode = GameMode.Shared,
            SessionName = "GLOBAL SERVER",
            SceneManager = NetworkRunner.GetComponent<INetworkSceneManager>(),
            IsOpen = true,
            PlayerCount = 3,


        }); ;
        Screen.SetResolution(width: 800, height: 450, fullscreen: false);//Just setting the resolution
        Application.targetFrameRate = 120;//Fps set to 120
    }

    public void OnConnectedToServer(NetworkRunner runner)
    {
        print(message: "Connected to server!");
    }
    public void OnPlayerJoined(NetworkRunner runner, PlayerRef player)
    {
        print(message: "Connected to session!");
    }


    public void OnPlayerLeft(NetworkRunner runner, PlayerRef player)
```

[Link To Source Code]

#8,2: Set Network Runner Prefab To Our ServerManager

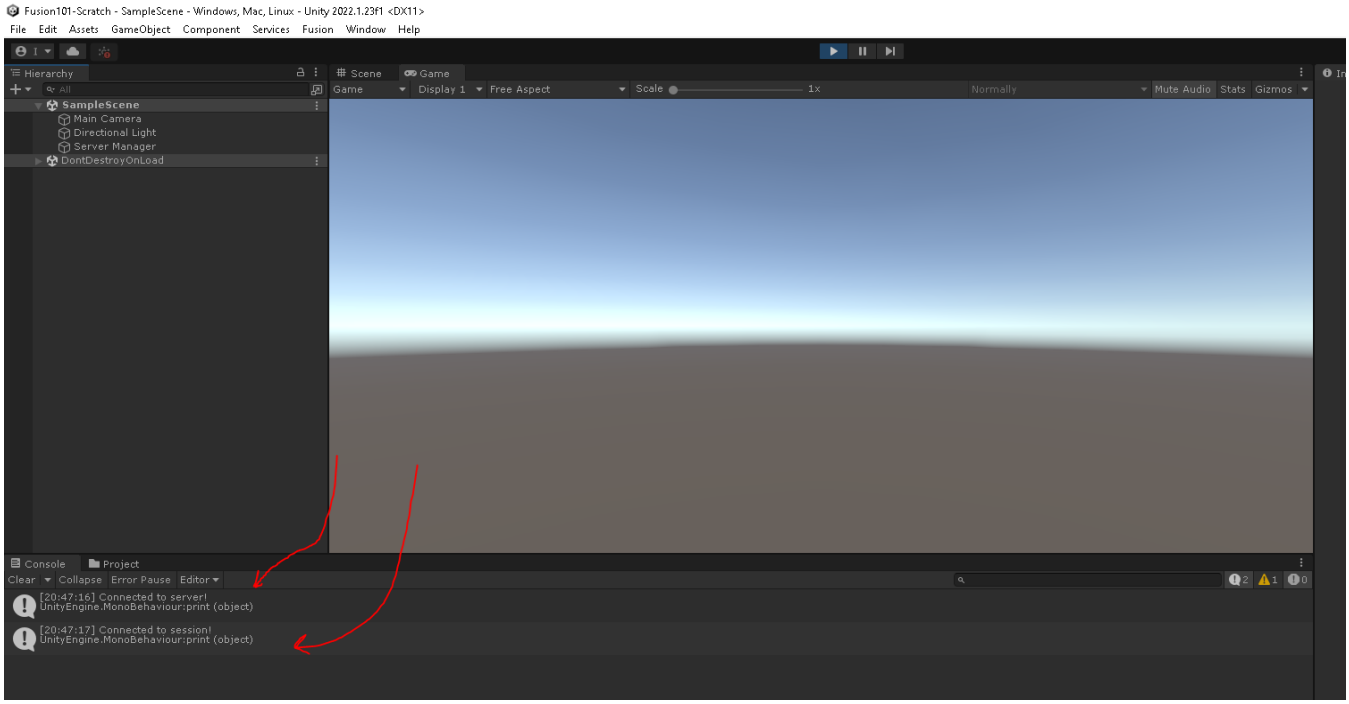


#9: Disable Debug Dlls

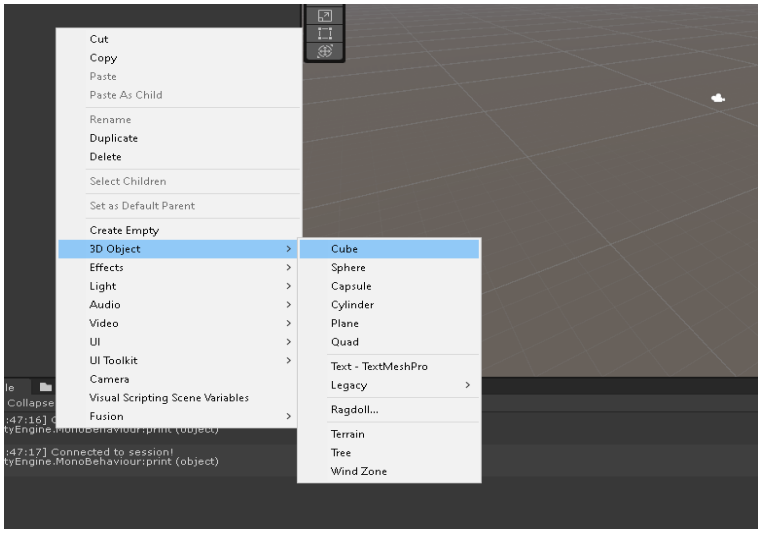


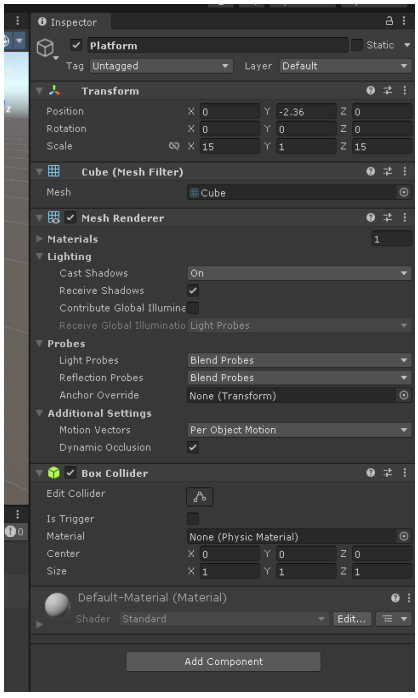
After implementing Fusion for the first time just press Toggle Debug Dlls once so we can clearly see our connection printouts. If you still see blue [Fusion] texts in the console Toggle Debug Dlls until it goes off.

#10: Start The Playmode & Check Connection

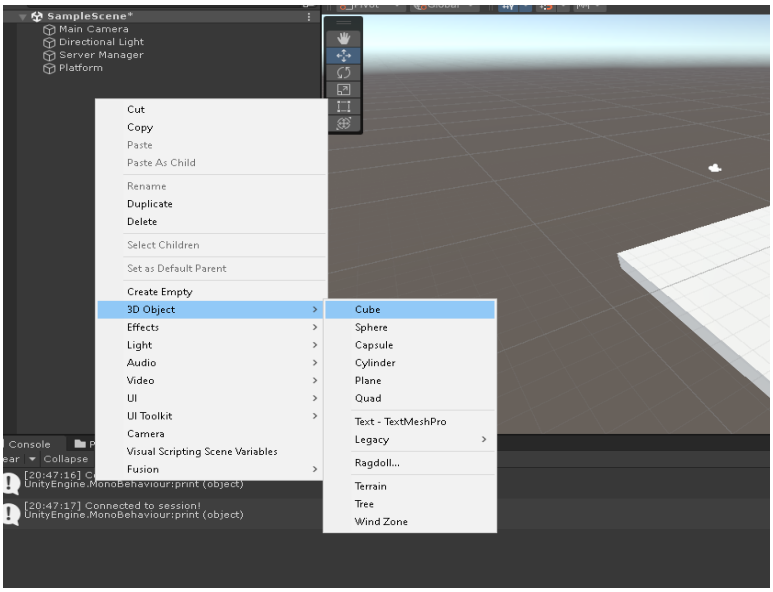


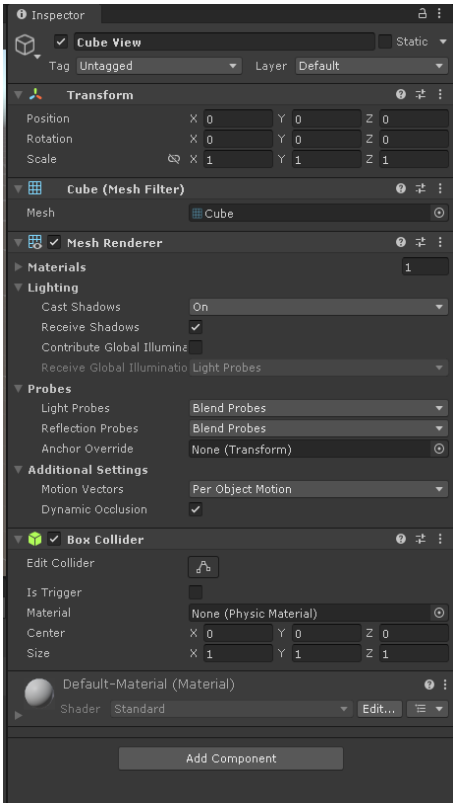
#11: Create A Platform In The Scene & Set It's Properties



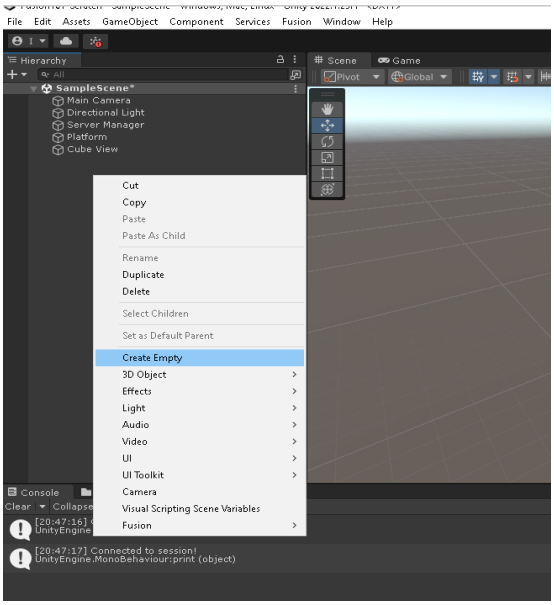


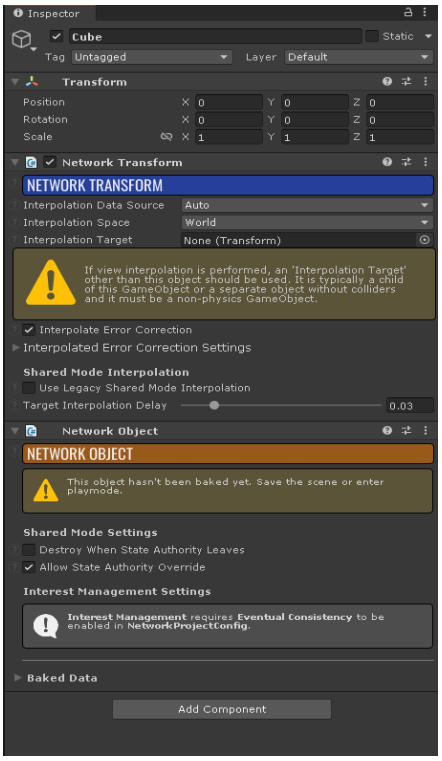
#12: Create A Cube “View” In The Scene & Set Its Properties



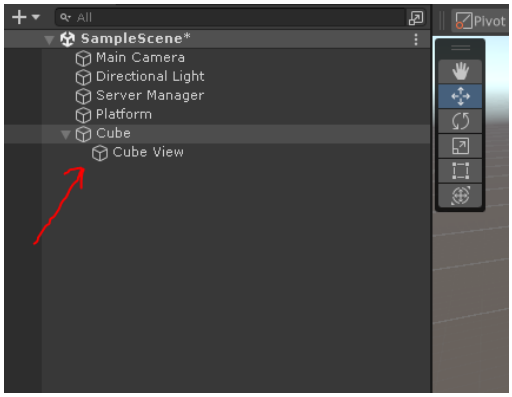


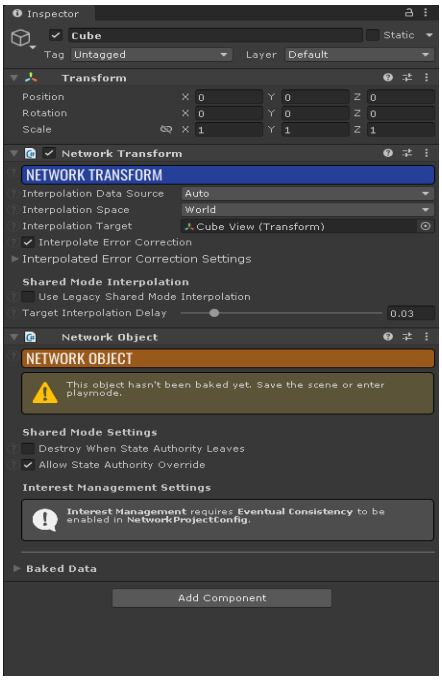
#13: Create A Cube In The Scene & Set Its Properties





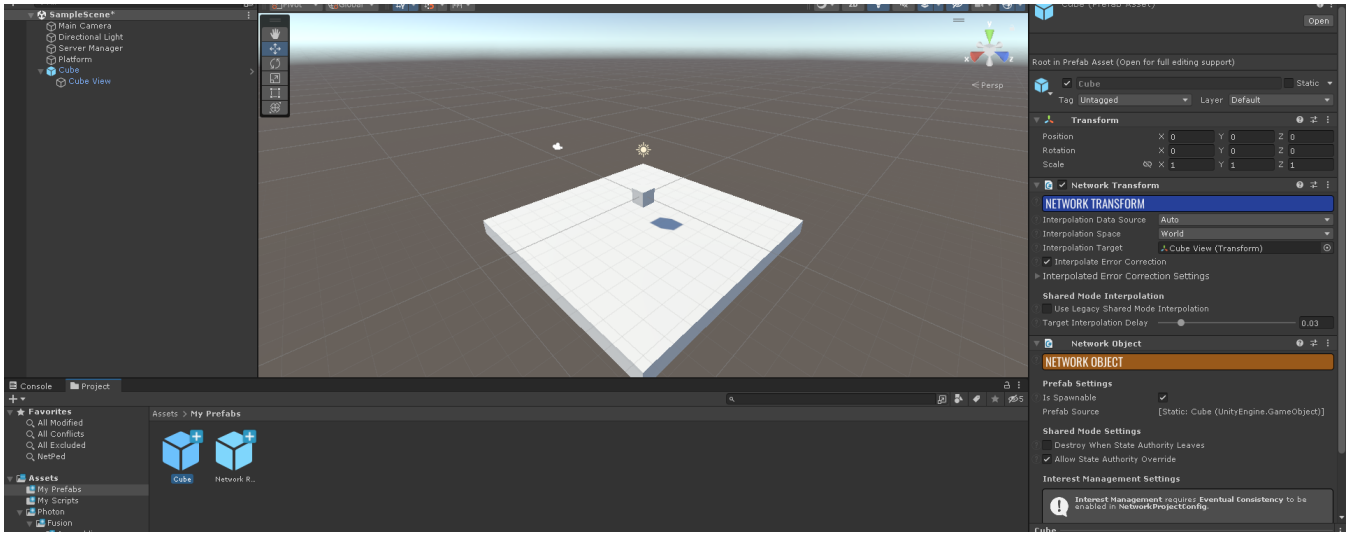
#14: Set Cube “View” Child Of Cube & Set Properties For Cube





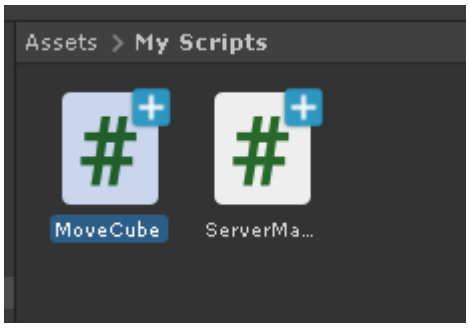
We will use Network Transform for Transform component to be synchronized.
Transform contains position, scale, rotation, etc. will all be synchronized in the simulation.

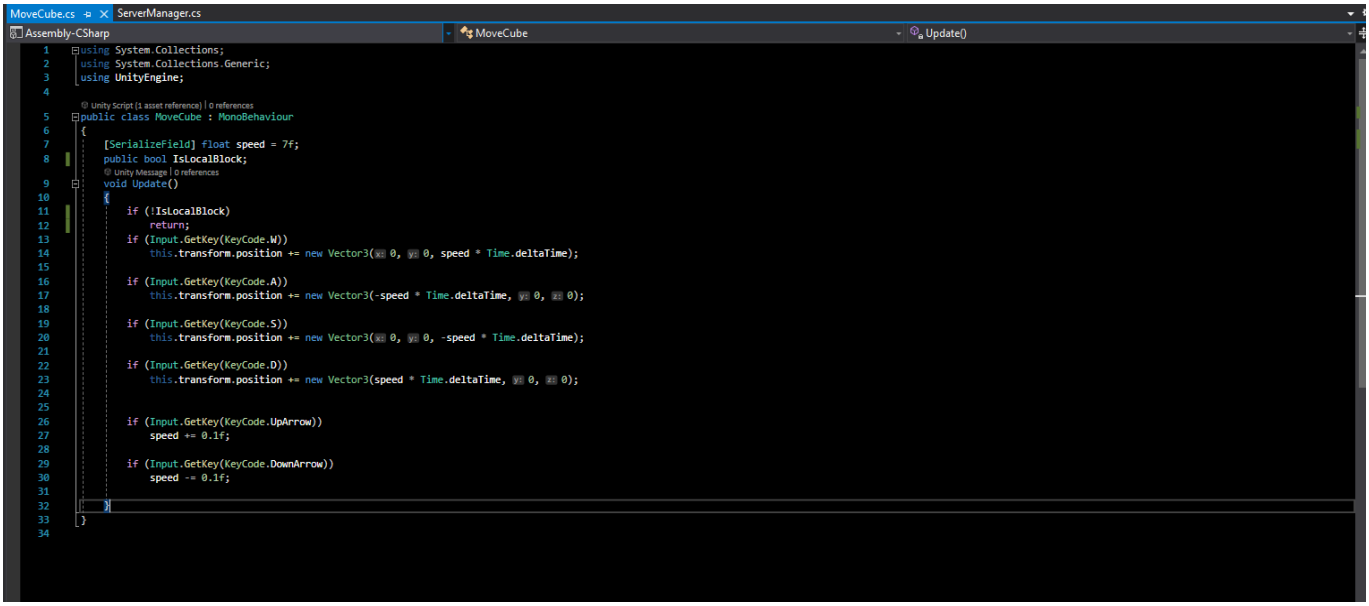
#15: Set Cube As Prefab And Delete It From Scene

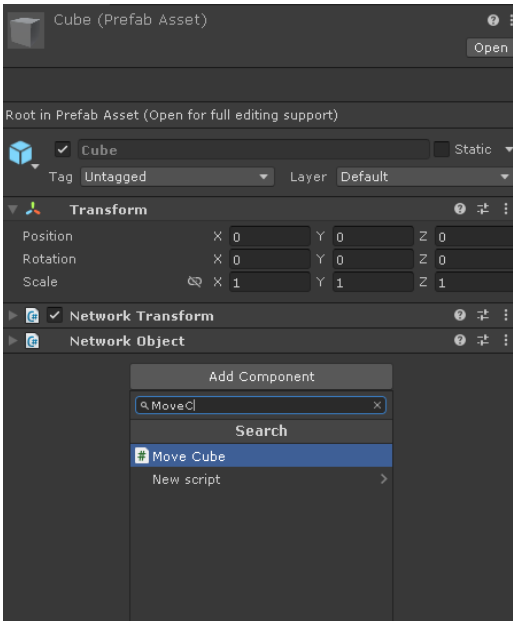


Network Objects are going to be spawned from prefab GameObjects.
Game Scene objects are not recommended.
View or whatever the model that is appearing on the screen sometimes it is Sprites it should be a child and its parent must have the Network Component and child must be assigned as Interpolation Target.

#16: Create MoveCube.cs & Script It & Add It To Cube [link to source code]





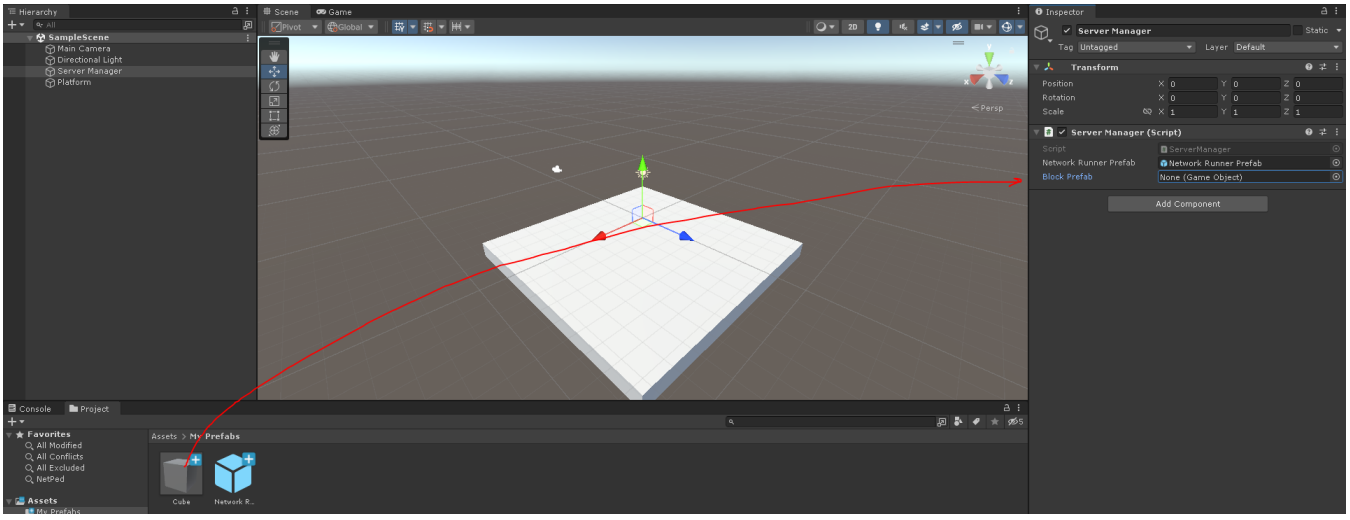


#17,1: Second Time Scripting ServerManager.cs: Spawn Cube As Network Object

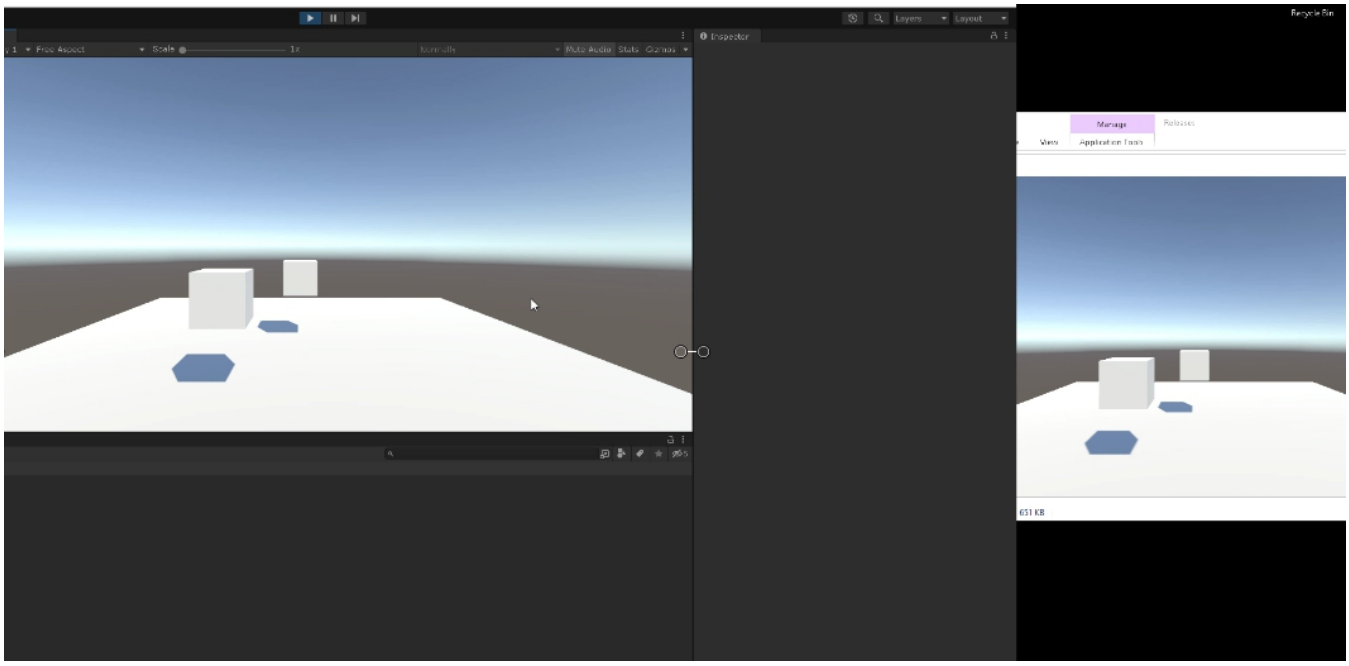
Link To Source Code

```
using System;
using Fusion;//IMPORTANT!
using Fusion.Sockets;//IMPORTANT!

Unity Script (1 asset reference) | 0 references
public class ServerManager : MonoBehaviour, INetworkRunnerCallbacks
{
    [SerializeField] GameObject NetworkRunnerPrefab;
    public static NetworkRunner NetworkRunner;
    [SerializeField] GameObject BlockPrefab;//NEW CODE

    Unity Message | 0 references
    void Start()
    {
        NetworkRunner = Instantiate(NetworkRunnerPrefab).GetComponent<NetworkRunner>();//Creating Network Runner
        NetworkRunner.AddCallbacks(this);
        NetworkRunner.StartGame(args: new StartGameArgs() //Connecting to the server and the session
        {
            GameMode = GameMode.Shared,
            SessionName = "GLOBAL SERVER",
            SceneManager = NetworkRunner.GetComponent<INetworkSceneManager>(),
            IsOpen = true,
            PlayerCount = 3,


        }); ;
        Screen.SetResolution(width: 800, height: 450, fullscreen: false);//Just setting the resolution
        Application.targetFrameRate = 120;//Fps set to 120
    }

    0 references
    public void OnConnectedToServer(NetworkRunner runner)
    {
        print(message: "Connected to server!");
    }
    0 references
    public void OnPlayerJoined(NetworkRunner runner, PlayerRef player)
    {
        print(message: "Connected to session!");
        if (player == NetworkRunner.LocalPlayer)                                    //NEW CODE
        {                                                                           //NEW CODE
            var NetworkObject myBlock = NetworkRunner.Spawn(BlockPrefab, Vector3.zero, Quaternion.identity);//NEW CODE
            myBlock.GetComponent<MoveCube>().IsLocalBlock = true;                   //NEW CODE
        }                                                                           //NEW CODE
    }
```

#17,2: Assign The Field Of Block Prefab



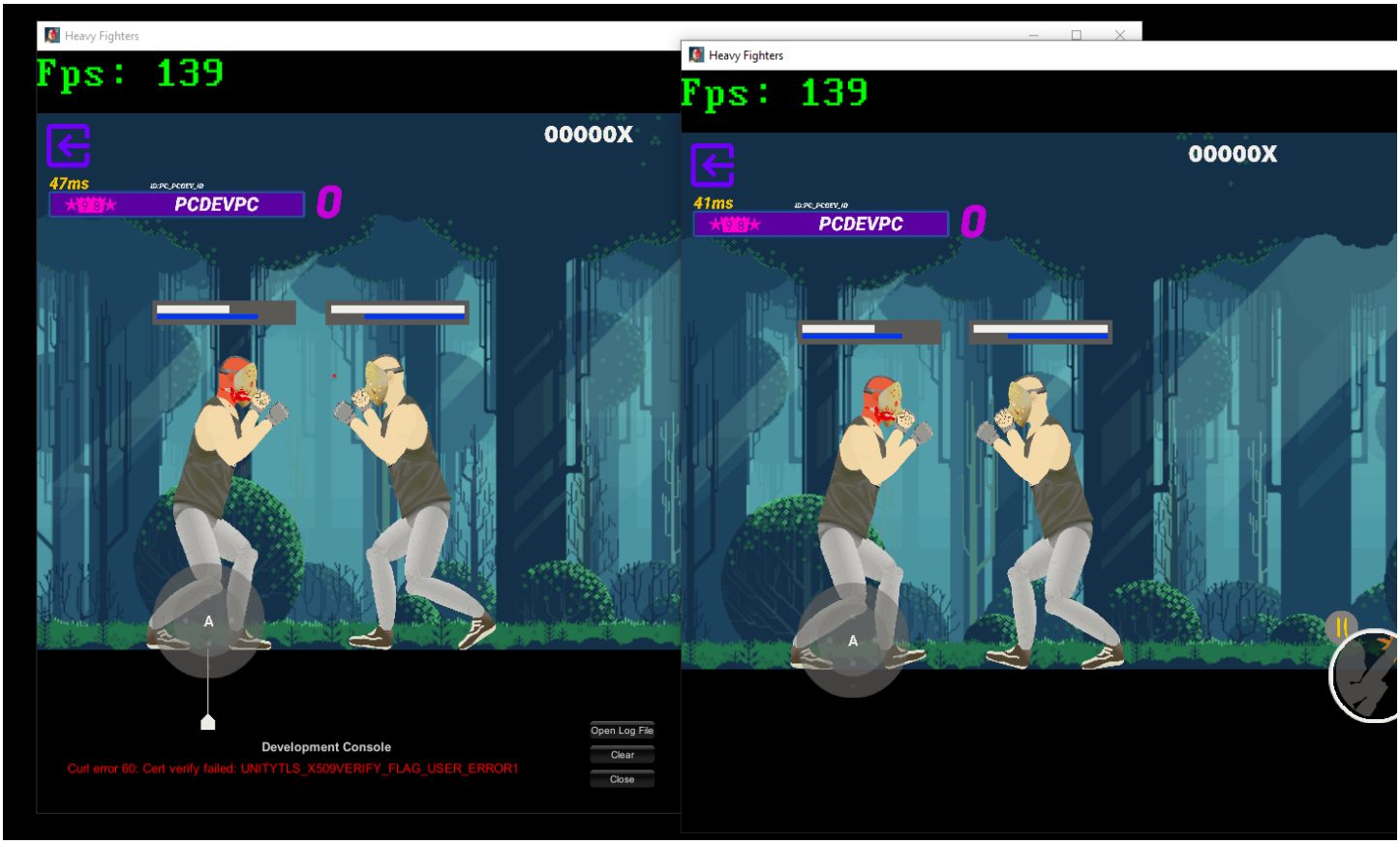
#18: Launching App With The Editor To Test Our Network!



(End Of The Basic Network Object Synchronization)

#Best Way To Learn
Change the scripts and their values!
Learn and test it by playing around!
You must code to learn.
You have to see that you can do things!
Don't get stuck with documentations.

#SHOWING
FINISHED PHOTON FUSION PROJECT
(Realtime Synchronization)
Heavy Fighters



#EXTENDED TOPICS

|

|

#EXTENDED TOPIC1:

#EXTENDED19: Using Network RigidBody From Our Cube Prefab
Remove Network Transform & Add Network RigidBody & Add Box Collider

Question1: Why lagging happens?
Hint1: Show the code
Answer:

#EXTENDED TOPIC2: Colouring The Blocks(Local Client Cube, Networked Cube)

#EXTENDED20,1: Update MoveCube.cs
source code

#EXTENDED20,2: Update For The Third Time ServerManager.cs
source code

Question2: Why colors of the blocks are different in two machines?
Answer: